Codex 深度体验报告

> 体验时间：2026-10 | 体验者：吕滢滢 | 广东金融学院 金融科技 说明：产品功能与价格以官网为准；本文结合公开资料与实际使用，重点分析产品策略与体验，不构成购买建议。

一、产品定位

- **一句话定位**：OpenAI 的软件工程 Agent——不是一个「会写代码的模型」，而是把模型包进「读仓库、改文件、跑命令、做审查」的任务执行系统，核心体验是「把任务派出去，再回来验收」。
- **目标用户画像**：
- 核心圈层：已经在使用 ChatGPT 的开发者——不需要再开一个新工具，同一个账号就能从「问代码」切换到「让 Agent 改代码」。
- 第二圈层：需要并行处理多个任务的工程团队——把互相独立的 bug、重构、测试补齐丢给云端并行跑。
- 第三圈层：希望把 AI 接入 GitHub 工作流的技术负责人——看重 PR 审查、权限、企业管理和沙箱隔离。
- **核心使用场景（3 个）**：
- 有边界的开发任务：修一个明确的 bug、补一组测试、做一个小功能，把任务描述清楚后交给 Agent 执行。
- 并行任务队列：同时开多个云端任务，分别处理不同分支/不同 issue，自己只负责验收和合并。
- 代码审查与原型验证：让 Codex 读 PR、找风险、提修改建议；或者快速做一个可运行的原型，再决定要不要投入。
- **市场位置 & 竞品**：Codex 的位置介于「编程 Agent」和「ChatGPT 生态入口」之间。它不像 Claude Code 那样强调终端里的深度定制，也不像 Cursor 那样把 IDE 作为主战场；它最强的是 **ChatGPT 的分发 + 云端并行 + GitHub 工作流**。核心竞品：Claude Code（深度工程协作）、Cursor（AI IDE）、GitHub Copilot（企业标配）、Gemini CLI（Google 生态）。
- **收费模式**：随 ChatGPT 订阅提供，Plus / Pro / Business / Enterprise 均可使用；也支持 API key 接入 CLI/SDK。需要注意：用 API key 的路径通常只保留本地能力，云端任务、GitHub 审查等能力需要订阅账号。

二、核心功能拆解

功能 A：多入口任务委派

- **用户流程**：在终端用 Codex CLI 描述任务 → Agent 读仓库、改文件、跑测试 → 需要更长时间时把任务转到云端 → 关掉电脑，任务继续跑 → 在 Web/手机上看进度和结果。
- **交互亮点**：
- 四个入口（ChatGPT、云端、IDE 插件、CLI）共享同一个账号和任务体系，用户不需要重新学习一套产品。
- 「本地开始、云端继续」是很实用的连续性设计：轻任务在本地快速迭代，重任务交给云端慢慢跑。
- 云端任务不依赖用户电脑开机，适合「下班前派任务，第二天看结果」的工作节奏。
- **待改进点**：
- 入口多但边界不够清晰：什么时候用 CLI、什么时候用云端、什么时候直接在 ChatGPT 里问，新用户容易混。
- 本地和云端的环境差异（依赖、密钥、网络、数据）需要用户自己处理，任务迁移不是完全无感。
- 云端任务的排队和启动时间不固定，紧急任务不如本地直接跑。

功能 B：云端并行与代码审查

- **用户流程**：连接 GitHub 仓库 → 选择任务/PR → Codex 在独立环境中跑多个任务 → 每个任务产出 diff/PR → 用户在 GitHub 里审查、评论、合并。
- **交互亮点**：
- 并行 Agent + 独立环境，是 Codex 最有差异化的能力：一个人可以同时推进多个任务，像多了几个「可并行的工程助手」。
- 自动 PR 审查把 Agent 嵌进了团队已有的 review 流程，而不是另起一个聊天窗口。
- 云端沙箱隔离降低了「Agent 在本地乱改文件」的风险，对企业来说更好交代。
- **待改进点**：
- 云端环境是 GitHub 中心化的，非 GitHub 仓库或复杂内网环境的支持有限。
- 沙箱限制网络访问，涉及私有依赖、内部服务、真实数据的任务需要额外配置。
- 并行任务多了以后，任务状态、成本、优先级缺少一个统一的管理视图。

功能 C：ChatGPT 一体化与工程配置

- **用户流程**：在 ChatGPT 里直接调用 Codex → 用 AGENTS.md 约定项目规则 → 通过 MCP 接外部工具 → 用 Skills/Plugins 复用工作流 → 在 Business/Enterprise 里统一管理权限和数据。
- **交互亮点**：
- 对已有 ChatGPT 用户，Codex 的迁移成本几乎为零：同一个账号、同一套订阅、同一个对话入口。
- AGENTS.md 让项目规则可以跟着仓库走，团队换人时不用靠口口相传。
- 企业版提供 SSO、权限、审计、数据控制等能力，具备从个人工具走向组织采购的基础。
- **待改进点**：
- 定制化生态比 Claude Code 窄：Hooks、Skills、Subagents 的成熟度和社区资产还有差距。
- AGENTS.md 与 Claude Code 的 CLAUDE.md 形成事实上的标准分裂，同时使用两套工具的团队要维护两份规则。
- 计费与额度不够透明：不同模型、不同任务、云端/本地的消耗规则不一致，用户很难预估。

三、做得好的地方

1. **把「写代码」变成「派任务」** — 产品视角
2. Codex 的产品叙事不是「我帮你补全下一行」，而是「你把任务说清楚，我去做」。这改变了开发者的工作方式：从「和 AI 一起写」变成「管理一个可并行的任务队列」。
3. 对团队来说，这意味着瓶颈从「写代码的速度」转移到「任务拆解、验收和合并」。Codex 的产品设计围绕这个新瓶颈展开：云端环境、并行任务、PR 审查、任务跟踪。
4. **ChatGPT 的分发优势是别人很难复制的** — 商业视角
5. Codex 不需要从零获取开发者：ChatGPT 已经有庞大的用户基础，Plus/Pro 用户升级到 Codex 的心理门槛很低。
6. 这种「聊天 + 工作 + 编程」的一体化，让 Codex 在获客成本上天然优于独立 coding agent。对 OpenAI 来说，Codex 也是把 ChatGPT 从「问答」推向「执行」的关键一环。
7. **云端沙箱 + 企业控制，解决了「敢不敢用」的问题** — 用户视角
8. 对个人开发者，云端沙箱意味着 Agent 不会直接动本地文件；对企业，独立的云端环境、权限和审计让采购更容易过合规。

9. 这是 Codex 相对 Claude Code 的一个明确差异：Claude Code 更像「本地的工程搭档」，Codex 更像「可管理的云端工程团队」。

坦诚说：Codex 的三个硬伤

1. **计划漂移是最大的工作流问题。** 很多用户反馈：给 Codex 一个明确的计划，它执行到一半会「自由发挥」，偏离原始需求。对生产代码来说，这意味着用户必须反复检查 diff、补充约束，省下的时间又被 review 吃回去。
2. **计费与额度不透明。** Codex 的消耗和 ChatGPT 套餐共享，不同模型、不同任务、云端/本地的计量规则不一致；用户很难在任务开始前知道「这次大概花多少」。API key 路径又会失去云端能力，选择成本高。
3. **定制化生态还不够深。** 相比 Claude Code 的 CLAUDE.md + Skills + Hooks + Subagents + Plugins 组合，Codex 的 AGENTS.md/MCP/Skills 还在追赶；对需要把 Agent 深度嵌入团队流程的用户，可塑性不足。

四、如果我是 PM，我会改进什么

1. **把计划变成可检查的「任务契约」** — 优先级：高
2. 理由：计划漂移的根源不是模型不聪明，而是「计划」只存在于自然语言里，没有变成可验证的检查点。
3. 改进方案：任务开始时把计划拆成 checklist（改哪些文件、跑哪些测试、验收标准是什么）；执行中每完成一项就勾选；偏离计划时主动提示「这一步不在原计划里，是否继续？」；最后用 checklist 做交付验收。
4. 预期效果：降低 review 成本，让用户敢把更长的任务交给 Codex。
5. **任务级成本预估与消耗明细** — 优先级：高
6. 理由：用户不是不愿意付费，而是不愿意「不知道会花多少」。不透明会直接抑制重度使用。
7. 改进方案：任务启动前给出预计消耗区间；执行中显示已用 token/额度；结束后给出按模型、按工具调用的明细；支持设置单任务预算上限。
8. 预期效果：减少「做到一半被限额打断」的失败任务，提升 Pro/ Business 的续费意愿。
9. **跨 Agent 的配置互操作** — 优先级：中
10. 理由：同时使用 Codex 和 Claude Code 的团队越来越多，AGENTS.md 与 CLAUDE.md 的分裂会增加维护成本。
11. 改进方案：支持导入/导出通用的 Agent 规则（至少兼容 AGENTS.md 和 CLAUDE.md 的核心字段）；提供「一份规则、多个 Agent 读取」的适配层；在 IDE 里提示两套规则的冲突。
12. 预期效果：降低多 Agent 团队的切换成本，也让 Codex 更容易进入已经使用 Claude Code 的团队。

五、竞品对比简表

维度	Codex	Claude Code	Cursor	GitHub Copilot
产品形态	CLI + IDE + 云端 + ChatGPT	终端为核心，多入口	AI 原生 IDE	IDE 插件 + GitHub 平台
核心优势	云端并行、ChatGPT 分发、沙箱	长任务、多文件重构、定制层深	内联编辑体验、多模型	GitHub 生态、企业采购
核心劣势	计划漂移、计量不透明	成本/额度、终端学习曲线	Agent 深度弱于专用工具	Agent 能力偏保守
目标用户	ChatGPT 用户、工程团队	重度开发者、技术负责人	广泛开发者	企业开发团队

维度	Codex	Claude Code	Cursor	GitHub Copilot
收费模式	随 ChatGPT 订阅 + API	Pro/Max 订阅 + API	订阅制	个人/企业订阅
一句话	最像「可并行的任务队列」	最像「工程搭档」	最像「AI IDE」	最像「企业标配」

六、一个启发

Codex 让我看到 coding agent 的竞争正在从「谁更会写代码」转向「谁更会让用户放心地派任务」。

模型能力会趋同，但「派任务」这件事包含三个产品问题：任务怎么描述、执行过程怎么监督、结果怎么验收。Codex 用云端并行和 PR 审查回答了「执行」和「验收」，但在「任务描述」和「过程监督」上还有明显空间——计划漂移、计量不透明，本质上都是「用户对任务失去控制感」。

这对 AI 产品经理的启发是：Agent 产品的价值不是「替用户做更多」，而是「让用户敢交出去更多」。信任不是靠模型分数建立的，而是靠可检查的计划、可回滚的操作、可解释的成本建立的。

七、运营视角拆解

拉新：ChatGPT 的存量用户是 Codex 最大的漏斗。运营不需要从零教育「什么是 AI 编程」，只需要在 ChatGPT 里让用户自然地从「问代码」过渡到「让 Codex 改代码」。

激活：首次价值时刻是「第一个 PR 被合并」，而不是「第一次生成代码」。因此 onboarding 应该引导用户从一个有明确验收标准的小任务开始，而不是一上来就丢一个大型重构。

留存：GitHub 仓库连接、云端环境、AGENTS.md、PR 审查记录都是留存资产；团队把 Codex 接入 review 流程后，留存从「个人习惯」变成「团队流程」。

商业化：订阅捆绑是 Codex 的最大优势——Plus/Pro 用户不需要额外付费就能尝试；Business/Enterprise 通过权限、审计、数据控制实现溢价。风险在于计量不透明会让重度用户产生「被限流」的挫败感。

运营痛点：计划漂移带来的口碑风险、额度不透明带来的付费摩擦、与 Claude Code 的配置分裂。运营需要把「并行」翻译成「可感知的产能」，而不是「更多需要 review 的 diff」。